

Procesador RISC-V Pipelined con Extensión C

Pipeline, Hazard Unit e Instrucciones Comprimidas

Ildefonso Santos
Steve Andy

Paucar Barrios
Miguel Luis

Rosillo Rodríguez
Marcelo Mateo

UTEC — Arquitectura de Computadoras

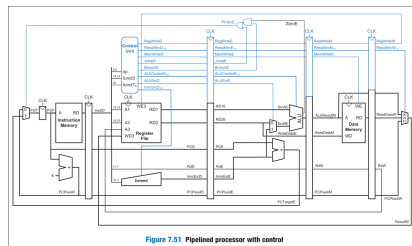
Julio 2026

- 1 Pipeline y Hazard Unit
- 2 Extensión C: instrucciones comprimidas
- 3 Resultados: algoritmo y comparativa
- 4 Conclusiones

- 1 Pipeline y Hazard Unit
- 2 Extensión C: instrucciones comprimidas
- 3 Resultados: algoritmo y comparativa
- 4 Conclusiones

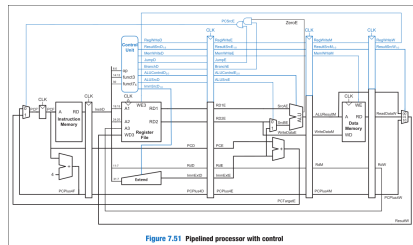
El pipeline base: 5 etapas

- **Fetch:** PC, memoria de instrucciones



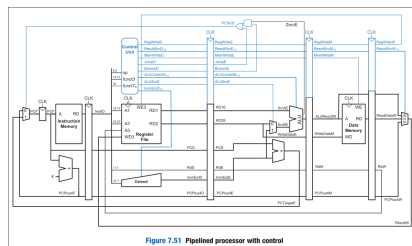
El pipeline base: 5 etapas

- **Fetch:** PC, memoria de instrucciones
- **Decode:** banco de registros, extend
- **Execute:** ALU, cálculo de saltos



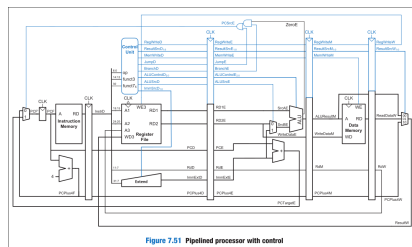
El pipeline base: 5 etapas

- **Fetch:** PC, memoria de instrucciones
- **Decode:** banco de registros, extend
- **Execute:** ALU, cálculo de saltos
- **Memory:** memoria de datos



El pipeline base: 5 etapas

- **Fetch:** PC, memoria de instrucciones
- **Decode:** banco de registros, extend
- **Execute:** ALU, cálculo de saltos
- **Memory:** memoria de datos
- **Writeback:** escritura al banco de registros

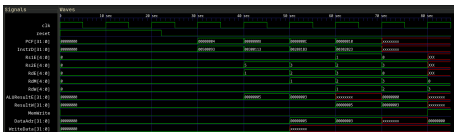


El precio de segmentar

Varias instrucciones en vuelo a la vez $\Rightarrow$ los datos y saltos no siempre están listos a tiempo.

- **De datos (RAW):** una instrucción usa un resultado que otra anterior aún no escribió. Caso crítico: *load-use*.
- **De control:** los saltos se resuelven en Execute, pero ya se captaron instrucciones secuenciales de más.
- **Estructurales:** evitados por diseño (imem/dmem separadas, regfile lee y escribe en el mismo ciclo).

Sin unidad de riesgos: los programas fallan



- add x3,x1,x2 lee x1/x2 **antes** de que se escriban en Writeback.
- ALUResultE sale en X.
- El dato corrupto llega hasta WriteData.

Solución: *forwarding, stall, flush*

- **Forwarding:** adelanta el resultado desde EX/MEM o MEM/WB directo a la ALU (ForwardAE/BE).

Resultado

Los mismos tres programas (forward, stall, flush) **pasan** correctamente con la hazard unit activa.

Solución: *forwarding, stall, flush*

- **Forwarding:** adelanta el resultado desde EX/MEM o MEM/WB directo a la ALU (ForwardAE/BE).
- **Stalling:** detiene Fetch/Decode un ciclo en el caso *load-use* (StallF/D).

Resultado

Los mismos tres programas (forward, stall, flush) **pasan** correctamente con la hazard unit activa.

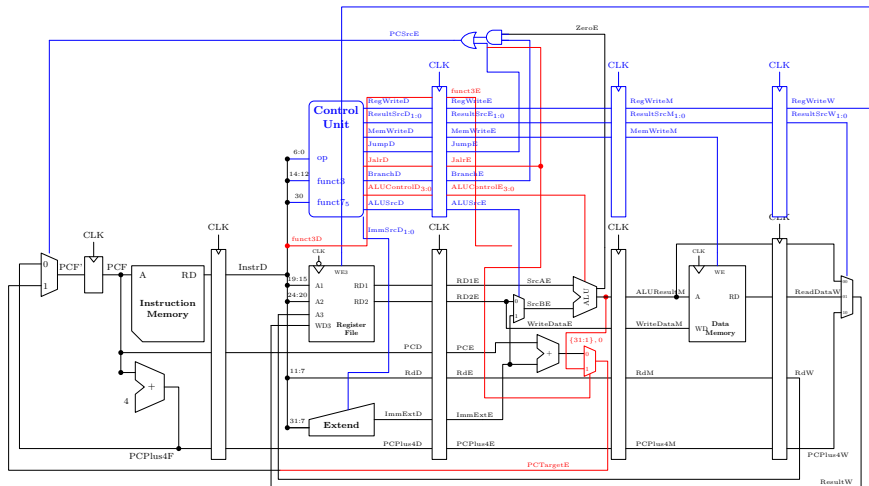
Solución: *forwarding, stall, flush*

- **Forwarding:** adelanta el resultado desde EX/MEM o MEM/WB directo a la ALU (ForwardAE/BE).
- **Stalling:** detiene Fetch/Decode un ciclo en el caso *load-use* (StallF/D).
- **Flushing:** descarta instrucciones mal captadas tras un salto tomado (FlushD/E).

Resultado

Los mismos tres programas (forward, stall, flush) **pasan** correctamente con la hazard unit activa.

Datapath final: implementación de instrucciones



En rojo: adiciones propias sobre el pipeline base — Jalr, funct3E, ALUControl1 de 4 bits, mux de destino jalr.

Contenido

- 1 Pipeline y Hazard Unit
- 2 Extensión C: instrucciones comprimidas
- 3 Resultados: algoritmo y comparativa
- 4 Conclusiones

¿Qué es RVC?

- Instrucciones de **16 bits** que coexisten con las de 32 bits.
- Reducen el tamaño del código **25–30 %** sin perder expresividad.
- Cada comprimida tiene **exactamente un** equivalente de 32 bits.

Estrategia: descomprimir en Fetch

Expandir los 16 bits a RV32I **antes** del registro IF/ID. Decode, Execute, Memory, Writeback, controller, hazardunit **no se modifican**.

Cambios en el datapath

- 1 **Ventana deslizante** en imem: una comprimida puede dejar al PC desalineado a palabra.

```
assign rd = a[1] ? {w1[15:0], w0[31:16]} : w0;
```

- 2 **Incremento variable**: $PC += \text{compressed} ? 2 : 4$.
- 3 **Módulo** decompressor.v: combinacional, produce la instrucción expandida antes del IF/ID.

Costo en ciclos

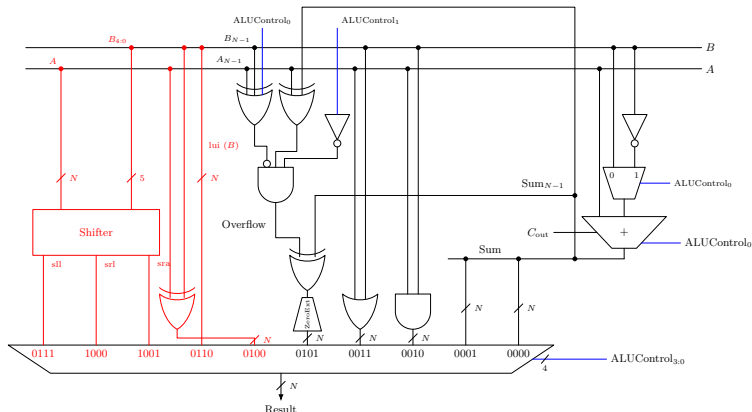
Cero. Todo es combinacional dentro de la etapa Fetch.

Módulos agregados / afectados

Módulo	Rol
<code>decompressor.v</code>	Nuevo. Expande 16→32 bits (combinacional)
<code>imem.v</code>	Ventana deslizante para PC desalineado
<code>datapath.v</code>	PCPlusInc, instancia del decompressor
<code>alu.v</code>	+5 operaciones: xor, lui, sll, srl, sra

Sin cambios: `controller`, `extend`, `regfile`, `hazardunit`, `dmem`

ALU modificada



Rojo: xor, lui, y el shifter (sll/srl/sra). Mux de salida creció de 5 a 10 entradas;
ALUControl de 3 a 4 bits.

Las 21 instrucciones comprimidas

Parte 1 (E2) — 11 instr.

- CL: `c.addi`, `c.lui`, `c.slli`
- CR: `c.add`
- CA: `c.sub`, `c.xor`, `c.or`, `c.and`
- CB: `c.srli`, `c.srai`, `c.andi`

Parte 2 (E3) — 10 instr.

- CL/CS: `c.lw`, `c.sw`
- CI/CSS: `c.lwsp`, `c.swsp`
- CB: `c.beqz`, `c.bnez`
- CJ: `c.j`, `c.jal`
- CR: `c.jr`, `c.jalr`

Ejemplo: c.lw / c.sw

Registros restringidos x8-x15, *offset* de 7 bits **barajado** (*scrambled*):

```
wire [6:0] offlw = {c[5], c[12:10], c[6], 2'b00};

5'b00_010: instr = {5'b0, offlw, rs1p, 3'b010,
                    rs2p, 7'b0000011};           // c.lw

5'b00_110: instr = {5'b0, offlw[6:5], rs2p, rs1p,
                    3'b010, offlw[4:2], 2'b00,
                    7'b0100011};                 // c.sw
```

Bug detectado y corregido: el destino de c.lw vive en c[4:2] (formato CL), no en c[9:7] — se confunde fácilmente con rs1'.

Ejemplo: desambiguación c.jr/c.jalr/c.add

Comparten {op,funct3}={10,100} (formato CR) junto con c.mv (subproducto de la desambiguación, no requerido). Se distinguen con c[12] y si rs2 es cero:

```
if (!c[12] && rs2 == 5'b0)
    instr = {12'b0, rd, 3'b000, 5'b00000, 7'b1100111}; // c.jr
else if (!c[12])
    instr = {7'b0, rs2, 5'b00000, 3'b000, rd, 7'b0110011}; // c.mv
else if (rs2 == 5'b0)
    instr = {12'b0, rd, 3'b000, 5'b00001, 7'b1100111}; // c.jalr
else
    instr = {7'b0000000, rs2, rd, 3'b000, rd, 7'b0110011}; // c.add
```

c[12]	rs2=0?	Instrucción	Expande a
0	sí	c.jr	jalr x0, rs1, 0
1	sí	c.jalr	jalr x1, rs1, 0
1	no	c.add	add rd, rd, rs2

Encoding de las 21 instrucciones

Instr.	op	f3	Fmt
c.addi	01	000	CI
c.jal	01	001	CJ
c.lui	01	011	CI
c.srli	01	100	CB
c.srai	01	100	CB
c.andi	01	100	CB
c.sub	01	100	CA
c.xor	01	100	CA
c.or	01	100	CA
c.and	01	100	CA
c.j	01	101	CJ
c.beqz	01	110	CB

Instr.	op	f3	Fmt
c.bnez	01	111	CB
c.slli	10	000	CI
c.lwsp	10	010	CI
c.jr	10	100	CR
c.jalr	10	100	CR
c.add	10	100	CR
c.swsp	10	110	CSS
c.lw	00	010	CL
c.sw	00	110	CS

$op = c[1:0]$, $funct3 = c[15:13]$

- `c.beqz/c.bnez` solo comparan contra `x0`: no existe compresión directa de `b1t/bge`.
- *Offsets* de memoria limitados: `c.lw/c.sw` hasta 124; superarlo obliga a materializar la dirección con instrucciones extra.
- Registros restringidos (`x8-x15`) en varios formatos: limita qué instrucciones se pueden comprimir según el registro usado.

Contenido

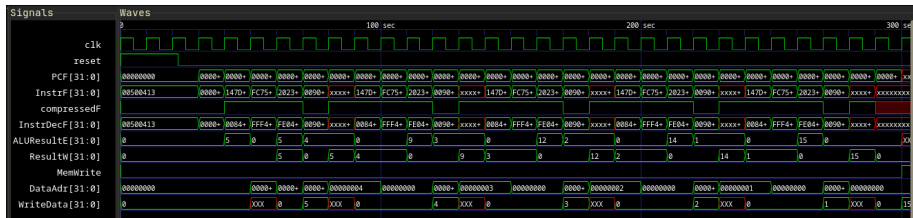
- 1 Pipeline y Hazard Unit
- 2 Extensión C: instrucciones comprimidas
- 3 Resultados: algoritmo y comparativa**
- 4 Conclusiones

Programa algoritmo: sumloop

Bucle que acumula $5 + 4 + 3 + 2 + 1 = 15$, mezclando 32 y 16 bits:

```
    addi x8, x0, 5      ; n = 5          (32 bits)
    addi x9, x0, 0      ; acc = 0        (32 bits)
LOOP:
    add  x9, x9, x8      ; c.add: acc += n (16 bits)
    addi x8, x8, -1      ; c.addi: n--   (16 bits)
    bnez x8, LOOP        ; c.bnez: repetir (16 bits)
    sw   x9, 0(x0)       ; Mem[0] = 15   (32 bits)
```

Waveform del algoritmo



PCF itera $0 \times 8 \rightarrow 0 \times A \rightarrow 0 \times C \rightarrow 0 \times 8$; ALUResultE acumula 5, 9, 12, 14, 15.

Comparativa: tamaño y *performance*

Versión	Instrucciones	Bytes	Reducción
RV32I puro	$6 \times 4B$	24	—
Con RVC	$3 \times 4B + 3 \times 2B$	18	25 %

Tamaño

25 % menos código. En programas reales, 25–30 % típico.

Performance

Idéntico: mismos ciclos, misma cantidad de instrucciones ejecutadas. La ventana deslizante no cuesta ciclos.

Categoría	Pruebas	Resultado
Pipeline base (E1)	12 testbenches	✓
RVC Parte 1 (E2)	caddi, cir, ca, cb, mul10	✓
RVC Parte 2 (E3)	clsw, clspsp, cbz, cj, cjr	✓
Algoritmo	sumloop	✓

23 / 23 PASS

Contenido

- 1 Pipeline y Hazard Unit
- 2 Extensión C: instrucciones comprimidas
- 3 Resultados: algoritmo y comparativa
- 4 Conclusiones

- **Conclusión clave:** descomprimir en Fetch permite añadir todo un ISA nuevo **sin tocar** el pipeline ni la hazard unit.

- **Conclusión clave:** descomprimir en Fetch permite añadir todo un ISA nuevo **sin tocar** el pipeline ni la hazard unit.
- **Desafío principal:** los *immediates* barajados (*scrambled*) — fácil confundir posiciones de bits entre formatos CL/CS/CB/CJ/CI/CSS.

- **Conclusión clave:** descomprimir en Fetch permite añadir todo un ISA nuevo **sin tocar** el pipeline ni la hazard unit.
- **Desafío principal:** los *immediates* barajados (*scrambled*) — fácil confundir posiciones de bits entre formatos CL/CS/CB/CJ/CI/CSS.
- **Mejora a futuro:** soportar *illegal instruction* trap para comprimidas mal formadas; extender a RV64C.

Gracias

Preguntas